

Actividad 1 - Regresión Lineal Simple

Idea: armamos un dataset sintético con una relación lineal conocida ($y = 2x + 3 + \text{ruido}$) y luego "fingimos" que no sabemos los coeficientes reales, para ver si MCO los recupera bien.

```
In [1]: import numpy as np

# fijamos la semilla para que el ejemplo sea reproducible
np.random.seed(42)

n = 100
X = np.random.rand(n) * 10          # variable predictora, valores entre 0 y 10
ruido = np.random.randn(n)         # error aleatorio ~ N(0,1)

# la relación real que queremos que MCO descubra: beta0=3, beta1=2
Y = 2 * X + 3 + ruido
```

Estimación por Mínimos Cuadrados Ordinarios

Usamos las fórmulas cerradas vistas en clase:

$$\hat{\beta}_1 = \frac{S_{xy}}{S_{xx}}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

```
In [2]: x_bar = np.mean(X)
        y_bar = np.mean(Y)

# Sxx: suma de cuadrados de X (mide dispersión de X)
Sxx = np.sum((X - x_bar) ** 2)
# Sxy: suma de productos cruzados (mide covarianza entre X y Y)
Sxy = np.sum((X - x_bar) * (Y - y_bar))

beta1_hat = Sxy / Sxx
beta0_hat = y_bar - beta1_hat * x_bar

print(f"beta0_hat (intercepto): {beta0_hat:.4f}")
print(f"beta1_hat (pendiente) : {beta1_hat:.4f}")
```

```
beta0_hat (intercepto): 3.2151
```

```
beta1_hat (pendiente) : 1.9540
```

Como los valores reales eran $\beta_0=3$ y $\beta_1=2$, y salimos con algo muy cercano, el estimador está funcionando bien (la diferencia es solo por el ruido aleatorio que agregamos).

Bondad de ajuste: MSE y R^2

```
In [3]: Y_hat = beta0_hat + beta1_hat * X    # valores predichos por el modelo
        residuos = Y - Y_hat

        SSE = np.sum(residuos ** 2)          # suma de cuadrados del error (varianza no expl
        SST = np.sum((Y - y_bar) ** 2)      # suma de cuadrados total (varianza total de Y)

        MSE = np.mean(residuos ** 2)        # error cuadrático medio
        R2 = 1 - SSE / SST                  # % de la variabilidad que el modelo logra expl

        print(f"MSE: {MSE:.4f}")
        print(f"R2 : {R2:.4f}")
```

MSE: 0.8066

R2 : 0.9765

Un R^2 cercano a 1 tiene sentido aquí porque generamos los datos con una relación lineal casi perfecta (solo con un poco de ruido normal encima).

Visualización

```
In [4]: import matplotlib.pyplot as plt

        plt.figure(figsize=(8, 5))

        # puntos originales
        plt.scatter(X, Y, color='steelblue', alpha=0.7, label='Datos observados')

        # recta ajustada por MCO (ordenamos X para que la línea salga bien trazada)
        orden = np.argsort(X)
        plt.plot(X[orden], Y_hat[orden], color='crimson', linewidth=2,
                 label=f'Ajuste: y = {beta1_hat:.2f}x + {beta0_hat:.2f}')

        plt.title('Regresión Lineal Simple (MCO)')
        plt.xlabel('X')
        plt.ylabel('Y')
        plt.legend()
        plt.grid(True, linestyle='--', alpha=0.5)
        plt.tight_layout()

        plt.show()
```

